Cache-Aside has the cache sitting *on the side* of the application, and only talk to the application. Differently from read-through, in this case it the responsibility of checking the cache or the DB is on the application, that will first check the cache and, in case of miss, do some extra work to also check the DB and store the value read on the cache.

Choosing the best strategy can be as important as acing the cache implementation, as choosing unwisely can overload and crash your database (or whatever the main storage is, in your case).

7.7 Introducing Synchronization

So far, we have always assumed that our cache objects were used in a single-threaded environment.

What do you think would happen if we used code in listing 7.6 in a concurrent environment? If you are familiar with concurrent code, you can already imagine: we might have issues with race conditions.

Listing 7.8 simplified version of LFU Cache set

```
function LFUCache::set(key, value)
  if (this.hashTable.contains(key))
then                                          #1
    node ← this.hashTable.get(key)
                                      #2
    node.setValue(value)
                                                  #3
```